UNIVERSITY
OF TWENTE.

June 17, 2025

M&M Sorting Machine

Akash Mondal
s3175626

David van Urk
s3276619

Abstract

For the course Engineering systems a winning sorting machine that sorts 25 *M&Ms* had to be made. The machine should be made of recycled materials and sort the *M&Ms* by color. The machine discussed in this report achieved a fastest time of 38 seconds, including a 3 second penalty for a faulty sorted *M&M*. In the competition, the sorting machine ended up in 3rd place.

Contents

1. Introduction 4
2. Design Specification 4
   2.1. Operational Requirements 4
   2.2. Functional Requirements 4
   2.3. Performance Requirements 4
3. Conceptual Design 5
   3.1. Prototypes 5
   3.2. Final Design 7
4. Detailed Design 10
   4.1. Physical Build 10
   4.2. Circuit 11
   4.3. Program 12
5. Testing 13
6. Conclusion 14
7. Appendix 15
   7.1. Code 15

1. Introduction

For the course Engineering Systems, we had to solve a problem. The problem here is sorting M&M's as fast as possible by color. In preparation for solving this problem we had a weekly tutorial to learn the language of *CircuitPython*, and the way it interacts with the Feather-M4 board. There are certain constraints we need to keep in mind, the first one of which is that it is only allowed to use recycled materials, such as carton, paperclips, old pipes, and other things laying around at home.

2. Design Specification

2.1. Operational Requirements

1. The machine has to be able to sort *M&Ms* reliably by color.
2. Has to be build stable so it won't fall over.
3. The building aspect needs to be done within a week from: June 24, 2024.
4. No money may be spend.
5. The team consist of two people.
6. Software must be build with the state-machine concept.
7. There must be a start and interrupt button
8. The machine must stop after 25 *M&Ms* are sorted.
9. The machine must print out the time it took for all *M&Ms* to be sorted.
10. Limit the amount of hot glue used on electronic components.
11. The machine must be able to sort any size and weight *M&M*.
12. The bin must indicate the expected color of the *M&M*.
13. The machine should be able to connect to the given filled M&M chamber.

2.2. Functional Requirements

1. The color of an *M&M* must be able to be measured.
2. The machine needs to put an *M&M* in the correct bin based on its color.
3. The *M&Ms* must be able to be inputted into the system.
4. The *M&Ms* must be transported from the input to the output.
5. The machine needs to be able to continue sorting a new *M&M* after an *M&M* has been sorted.

2.3. Performance Requirements

1. The machine has to be able to sort and output the *M&Ms* within a second per *M&M*.
2. The machine should be made of mostly recycled materials.
3. At most 2 servo's are allowed to be put in to the machine.
4. The machine should be a winning design.

3. Conceptual Design

While designing we decided to create and think of prototypes and test if they would work. This is because we liked a hands on approach where we can really see how the M&M's would behave. For example, we did assume that they are not perfectly round and they vary a lot in size. These prototypes were kind of rushed, this had to be done in order to be finished in time (Operational Requirement 3 and 5). So if something also did not seem like a viable option to investigate further we decided to skip over it.

3.1. Prototypes

At first we thought we would want to use a single servo that has 2 doors, one for letting M&M's roll in, and another one that drops them down when the the *M&M* has been assigned a color. This way, inputting and outputting the *M&M* would be parallelized, trying to meet Performance Requirement 1 and 4. This prototype went off the table fast due to the number of *M&Ms* inputted being quite unreliable because it came down to timing the servo so that it would only allow for precisely 1 *M&M* to go through. Further more, turning a rectangular piece of carton that is already snug with the surface next to it results in a lot of friction. The first prototype can be seen in Figure 1.

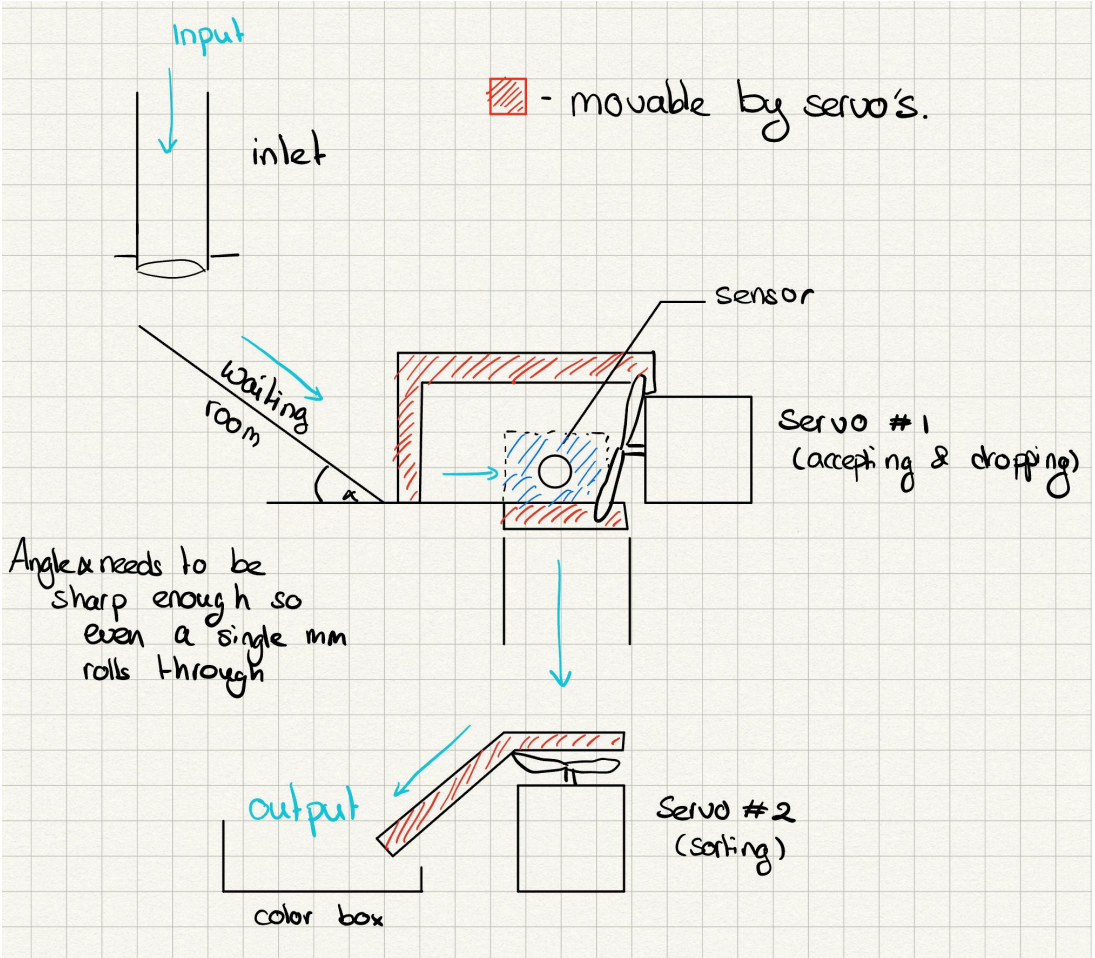


Figure 1: Prototype of parallelized input-output system.

Building upon the idea of the first prototype of parallelization, the second idea was a sort of half-moon shaped piece of carton attached to the servo. On this piece of carton, there was a maze-like structure made of carton, which admits one *M&M* to be outputted when the servo turns. When the servo turns back only one *M&M* should have been let through. Two bumps were added to the input of the system which fitted 1 *M&M*, so that every turn only one *M&M* would be let through. However, small proof-of-

concept trials for the bumps did not work at all. The structure was also way too heavy for the servo to hold onto, resulting in this prototype being quickly scrapped. Fixing the heavy weight would require adding extra supports or an axle to the rotational system, but still a more powerful motor would be needed to meet Performance Requirement 1. However, this would not adhere to Performance Requirement 3. The design for this prototype can bee seen in Figure 2.

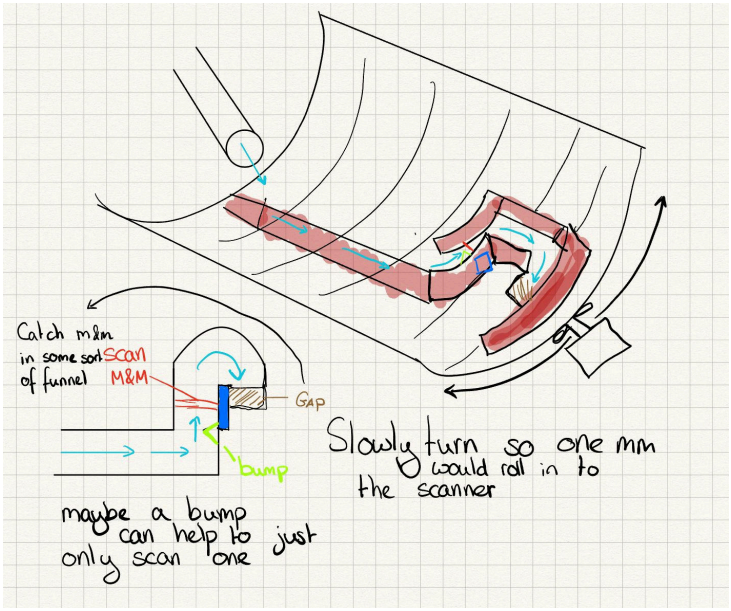


Figure 2: Prototype of the maze-like rotational input-output system.

The third prototype was a sort of boiled down version of the second prototype, but with less and smaller components to reduce the load on the servo. The piece of carton attached to the servo would turn to let an *M&M* go from the input to the output. Before turning, the *M&M* would be measured Turning the servo back to its original position at precisely the correct time, only one *M&M* would go through. After turning the servo back, the *M&M* that just when from input to output is outputted. This design also tries to parallelize the input and output, but suffers just like the first and second prototype from timing issues. The prototype can be seen in Figure 3.

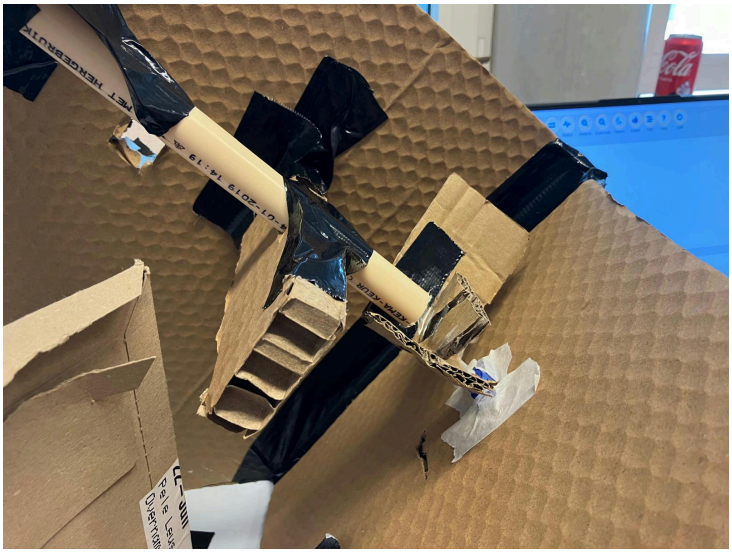


Figure 3: Prototype of smaller rotational input-output system.

3.2. Final Design

After spending one and a half day on prototyping on roughly the same design based on parallelization which did not seem to be working. This is why we came up with a new design, which ultimately become our final design for the machine.

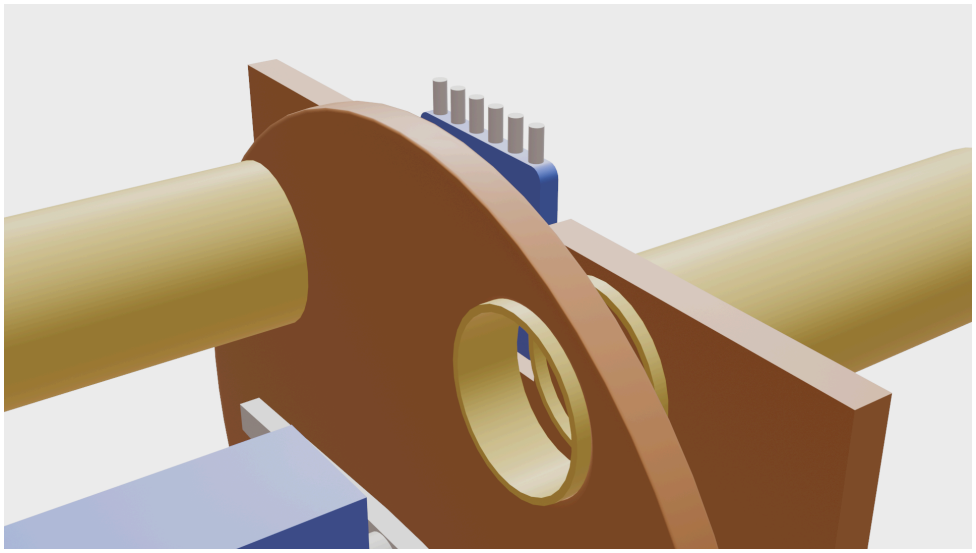


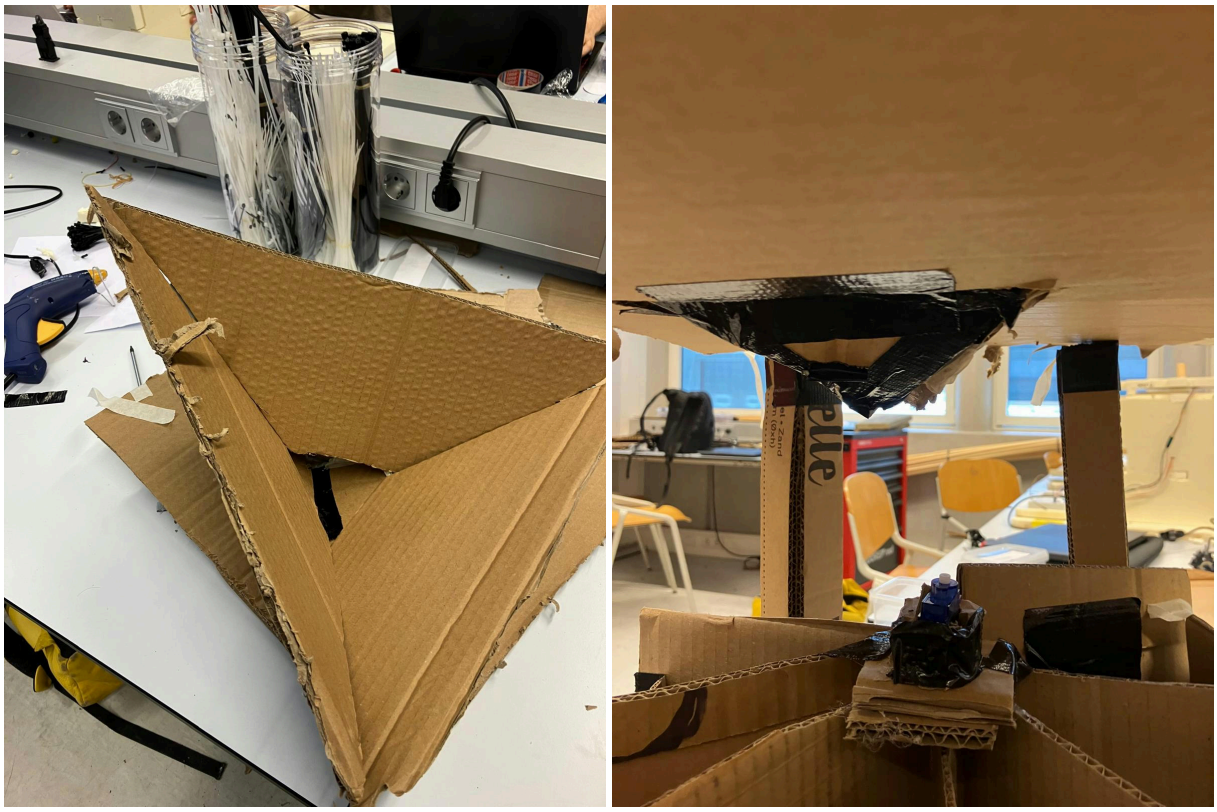
Figure 4: 3D sketch of separating and measuring *M&Ms*.

he mechanism works by using a wheel which lifts/rotates an *M&M* against a wall to the color sensor (see Figure 4). This way only a single *M&M* can be picked up each time, ensuring the reliability of the system and in turn meeting Operational Requirement 1. The reason the *M&M* is not scanned at an exit of a pipe or inside a pipe was because the PVC could reflect the light of other *M&Ms*, which would make the sensor less reliable. The wall that is in contact with the wheel and where the lies against can bend a little. This way heavier *M&Ms* will push the carton away a bit so they can also fit inside the wheel, while lighter and smaller ones can still go solo in to the wheel, ticking off Operational Requirement 11. Then the *M&M* rotates a little bit further where it rolls down. Because we wanted the scanning to be even more reliable we decided to use black tape as "wallpaper". Thanks to this separate chamber we were able to bring the integration time of the sensor to a minimum, while still getting great outputs. Even though the integration time was low, by checking the *M&Ms* in a separate chamber instead of directly we lost some speed. At first our machine could not feel or see when a *M&M* was loaded on we had to keep going back and forward from the input to the sensor to the output, which costs a lot of time. That is why we wanted to optimize this part as much as possible while keeping the great accuracy of the sensor. We gave the sensor some code that can check whether it is really an *M&M* or that it is measuring no *M&M*, in this case some carton. By doing this we could check that when no *M&M* was found inside the chamber we could go back to the input directly (instead of first going to the output). We also sped up the grabbing and outputting of the *M&Ms*, while trying to keep as much accuracy as possible. This process was a lot of trial and error. These optimizations caused us to get closer to Performance Requirement 1 and 4. Because the cardboard wall to which the sensor is attached is quite thin and the whole system is angled towards the output, the *M&M* that is being measured is ensured to be close enough to the sensor. Further more the *M&M* cannot be directly against the sensor. That is why a transparent piece of tape has been put in front of the sensor, against which the *M&M* lies. The sensor itself is placed a small distance from the piece of tape.

While that mechanism was being built we decided we would split up tasks where one would also build the part where *M&Ms* would be put in their respective box. This was done by building a "gutter" which would be facing downward and could be put on to a servo so that it can rotate to different directions and let the *M&Ms* fall down in to those directions (see Figure 8). Then by simply putting boxes beneath

the gutter we created a mechanism that can put each *M&M* in their respective box by changing the angle of the servo. This way both of our servos were used, which ticks Performance Requirement 3.

Because the two parts worked quite well separately we decided to build a simple funnel which would catch the outputted *M&Ms* (see Figure 5), without really taking the time restraints into consideration. This meant that the process kind of worked, but it was definitely not fast. Further more a considerable amount of *M&Ms* got stuck in or fell out of the funnel. That is why we decided we had to break the prototype scanning part of the machine a bit apart so the M&M's could directly fall in to the "gutter" instead of letting them fall down in to a funnel that can load them on to a gutter. This was done by putting a pipe in contact with the output of the scanner, where it would lead the *M&M* down in to the gutter. We still kept a little bit of height between the scanning and sorting system, for the *M&M* to pick up a little bit of speed in the pipe so that it has enough velocity to end up in one of the bins. On top of that the extra time that the *M&M* spends in the pipe gives a little bit of time for the sorting servo to turn to the correct bin, fulfilling Operational Requirement 1. Each box was labeled according to color in order to satisfy Operational Requirement 12. Because the speed was quite high we had to create some walls that can block the *M&Ms* so that they do not fall out of the system.



a) Catching part of the funnel b) Outputting part of the funnel

Figure 5: Funnel for taking *M&Ms* from measuring to sorting system.

The whole setup has been built using carton, tape and glue. This is because these materials are quite durable and if applied together effectively, quite rigid and stable as well. This ticks the second Operational Requirement. PVC tubing is also used because these are quite convenient for moving *M&Ms* around. All of these items were offered for free by the university or are recycled. This ticks Operational Requirement 4 and Performance Requirement 2. We decided to use duct tape to get attach the servo's to the cardboard instead of using hot glue due to Operational Requirement 10.

The full idea of the system can be seen in the diagram below (Figure 6):

1. The *M&M* is getting loaded in to the circular lift and moves to the scanner.
   - Functional Requirement 3.

2. The *M&M* does get scanned and checked whether it is there, if not then go back to the input.
   - Functional Requirement 1 & 2
3. If found, change the angle of the second servo according to the color and move the *M&M* towards the second servo with gutter. And load a new one in to the system.
   - Functional Requirement 2 & 5

By completing the cycle Functional Requirement 4 can be ticked of.

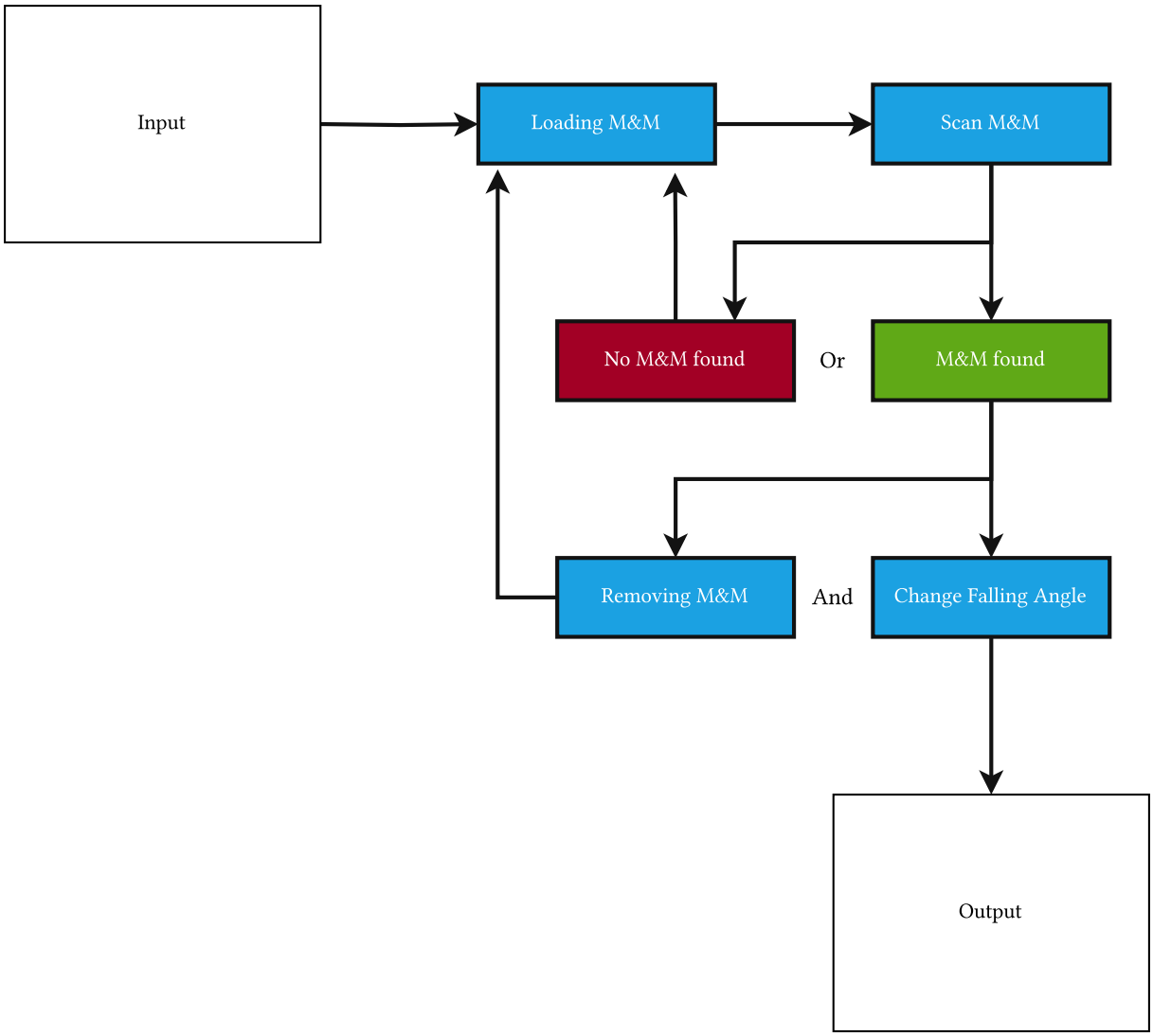


Figure 6: A diagram representing the idea of the system, where input is where the *M&Ms* are loaded in and the output is where the *M&Ms* finally drop out.

4. Detailed Design

4.1. Physical Build

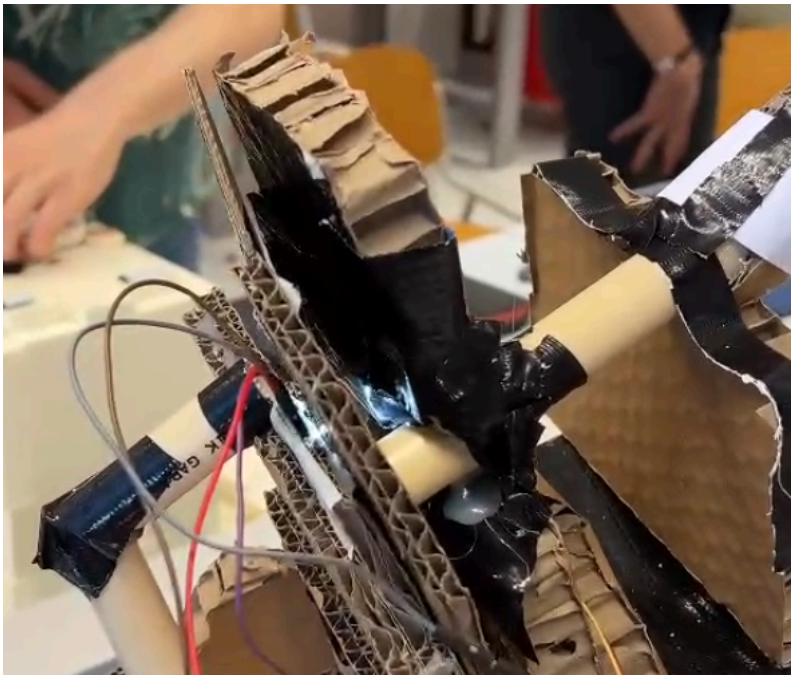


Figure 7: 3D sketch of separating and measuring *M&Ms*.

The final design can be seen in Figure 7. Here, the left tube lets the *M&Ms* in to the system by moving the wheel into the correct position. When the wheel turns the *M&M* that is now inside the wheel towards the sensor, no new *M&Ms* are admitted by the system because the cardboard wheel blocks new *M&Ms* to come in. The sensor is placed between the input and output tube. If the sensor measures that there is not *M&M* present, the wheel moves back to the input tube. If it does measure that an *M&M* is present, the wheel moves to the output tube, after which it returns back to the input tube, fulfilling Functional Requirement 5. The input tube is the same kind of PVC tube that is used at the competition to provide the 25 *M&Ms*, which makes it easy to connect them. This fulfills Operational Requirement 13.



Figure 8: Sorting gutter attached to servo.

When the *M&M* drops out of the out of the measuring system, a gutter (as can be seen in Figure 8) will move it towards the correct section based on the colors measured by the sensor.

4.2. Circuit

In Figure 9 the circuit that we used in our system can be seen. The Featherboard M4 was put in some sort of station where it was powered together with the servo kit. The button is used to start and stop the system.

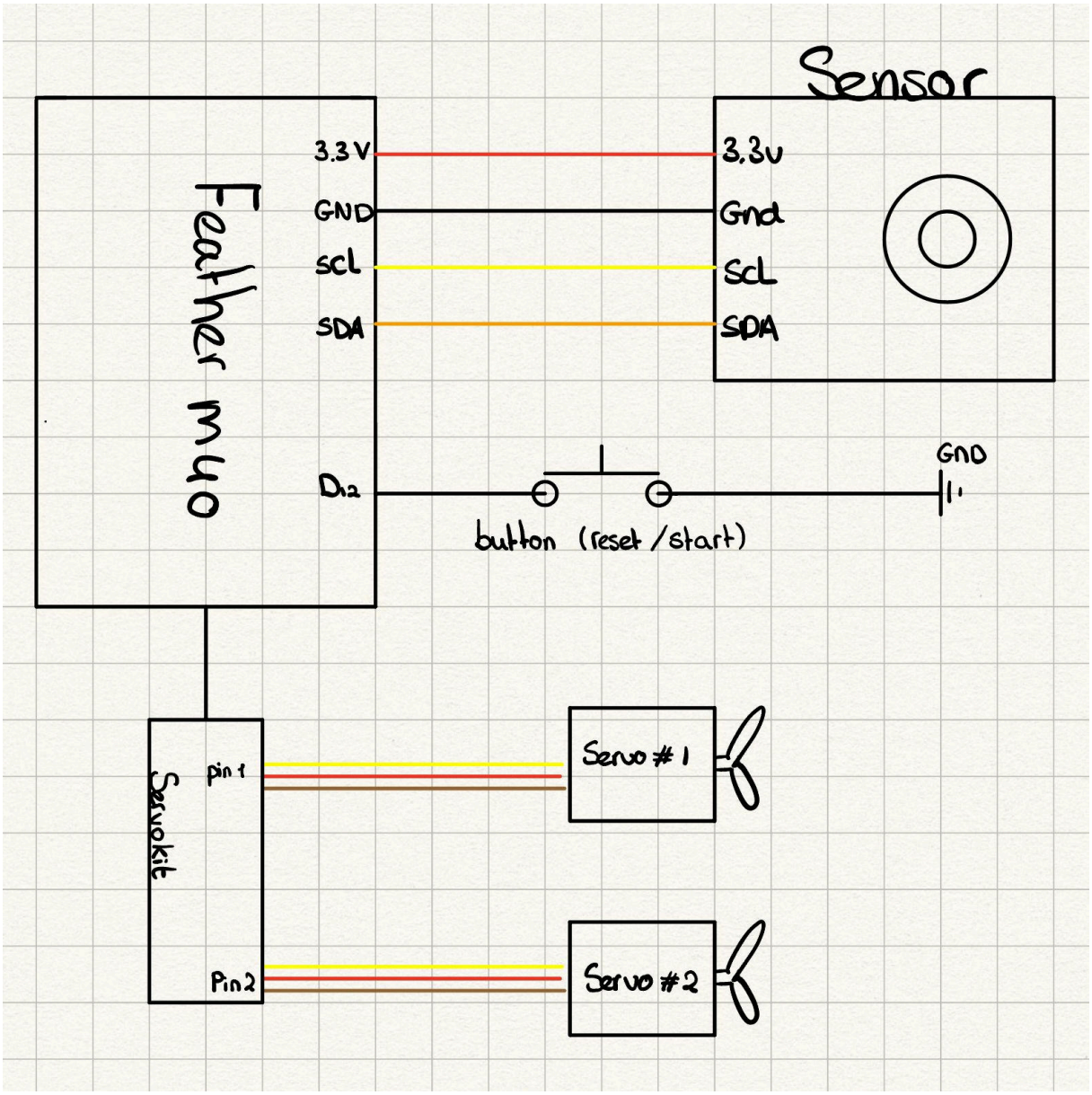


Figure 9: Electric circuit of the complete system.

4.3. Program

The program had to be coded with a state machine concept in mind (as can be seen in Figure 10), in compliance with Operational Requirement 6. A start/stop button was also implemented in the system in order to complete Operational Requirement 7. When looking at the code below there can be seen that at the `State.DATA` state the machine will print the time taken when all 25 *M&Ms* are sorted, and that the machine will be stopped, which completes Operational Requirement 8 and 9. We hardcoded the RGB, temperature and lux values in to the code instead of using A.I. because the amount of training would probably take way longer than is needed. Moreover most lights in the university are the same, and this is where our machine had to operate. The chance of our measured values being very different from room to room was not that big.

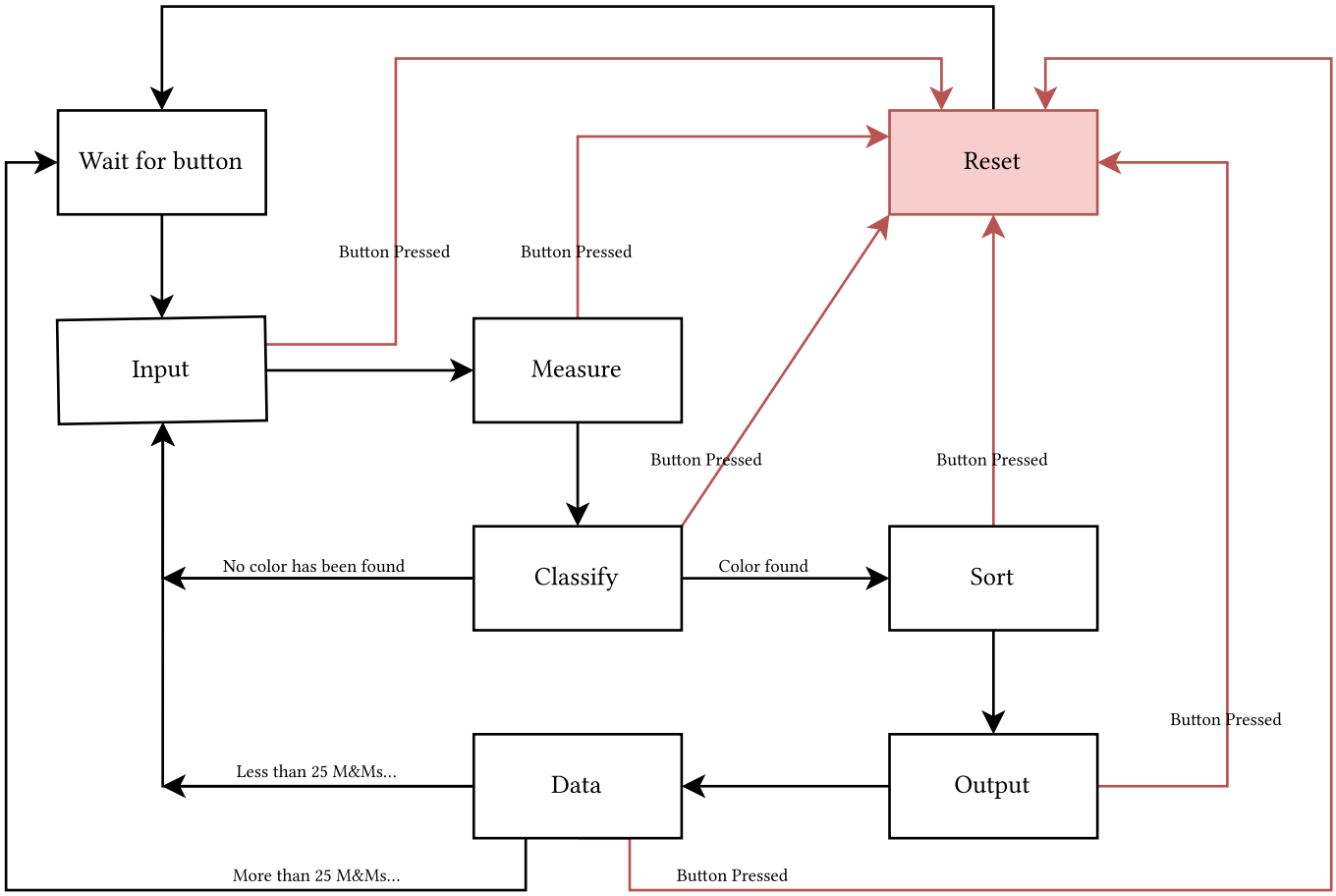


Figure 10: A diagram which describes the states used in our code (Section 7.1).

5. Testing

Several tests have been performed with our machine. The firsts few tests was testing various proof-of-concept mechanisms for our prototypes, which did not work most of the time. For the second prototype we noticed that *M&Ms* were still to hard to control due to their shape not being perfectly round. This caused them to get stuck or fall out of the container. The servo could also not reliably hold the weight of the carton.

When our final design came together we did several tests to check whether the principle was viable. Having a DNF is worse than making a slow-sorting mechanism, it is a sorting competition afterall. That is why we put our focus on getting the machine to work instead of it being fast, so we only ran 4 full timed test runs. All tests had 25 *M&Ms* loaded into the system. Our other tests were on the competition where it seemed that our machine did quite well. We managed to get to the top of the leaderboard by having a time of 38 seconds including a 3 second penalty for one faulty sorted *M&M*. The results of each test can be seen in Table 1.

What may seem odd is that the last result has way more wrong sorts than any other test, this is because we accidentally disconnected the gutter-servo, which caused almost all of the *M&Ms* to go into the same bin. Noticing the trend where the time taken sorting or the the amount of faulty sorts decreases. This indicates an improvement of our machine during the process of building and refining the measure and sorting systems.

Test N	**Time** in seconds	**Wrong sorts**
Before competition		
#1	DNF	-
#2	94	1 yellow in orange 1 brown in orange
#3	52	1 yellow in green 1 yellow in blue 1 red in blue
#4	54	1 yellow in green
During competition		
#5	47	No wrong sorts
#6	35	1 yellow in green
#7	40	15 wrong sorts No color checks.

Table 1: Test being performed on our system the numbers go from the first test up and including the last test. Time was measured without the penalty.

6. Conclusion

During one week a *M&M* sorting machine was built. The machine ended up working quite well which landed us in 3rd place at the sorting competition. It worked by separating a single *M&M* from the input, then letting it go in to a wheel, where it was scanned and then released in to an output. Our fastest measured time was 35 seconds, where one *M&M* was sorted wrong. This was done while meeting all the requirements mentioned in Section 2 of this report.

7. Appendix

7.1. Code

```
import time
from adafruit_servokit import ServoKit
import adafruit_tcs34725
import board
import busio
import digitalio

# Servo and sensor setup
MEASURE_SERVO = 1
SORTING_SERVO = 2
i2c = board.I2C()
sensor = adafruit_tcs34725.TCS34725(i2c)
kit = ServoKit(channels=8, i2c=i2c)

kit.servo[SORTING_SERVO].set_pulse_width_range(200, 2800)
kit.servo[MEASURE_SERVO].set_pulse_width_range(200, 2800)

sensor.integration_time = 2.4
sensor.gain = 1

# Button setup
button = digitalio.DigitalInOut(board.D12)
button.direction = digitalio.Direction.INPUT
button.pull = digitalio.Pull.UP

# Define states
class State:
    WAIT_FOR_BUTTON = 0
    INPUT = 1
    MEASURE = 2
    CLASSIFY = 3
    SORT = 4
    OUTPUT = 5
    DATA = 6

current_state = State.WAIT_FOR_BUTTON
next_state_time = time.monotonic()
output_count = 0
start_time = None

def set_sorting_angle(color):
    angles = {
        "Blue": 15,
        "Green": 35,
        "Yellow": 60,
        "Orange": 90,
        "Red": 120,
        "Brown": 150,
        "Nothing": 90
    }
     angle = angles.get(color, 90)  # Defaults to 90 if a non-existent color is
inputted
```

```
    kit.servo[SORTING_SERVO].angle = angle
    print(f"Set sorting angle to {angle} for color {color}")

def set_measure_servo_angle(loc):
    angles = {
        "Input": 56,
        "Measure": 80,
        "Output": 110
    }
    angle = angles.get(loc, 0)
    kit.servo[MEASURE_SERVO].angle = angle
    print(f"Set measure servo angle to {angle} for location {loc}")

def classify_color():
    red, green, blue = sensor.color_rgb_bytes
    lux = sensor.lux
    temp = sensor.color_temperature

    if red >= 25 and lux > 1500 and lux < 3000:
        return "Red"
    elif green > 18 and lux > 3000:
        return "Green"
    elif blue > 15 and temp > 8000:
        return "Blue"
    elif red > 20 and green >= 15 and lux > 3500 and temp > 3000:
        return "Yellow"
    elif red > 25 and lux > 2500:
        return "Orange"
    elif red > 15 and lux > 1500:
        return "Brown"
    else:
        return "Nothing"

# Reset counter if button pressed during measurement
def button_reset(State):
    global output_count, start_time, current_state
    output_count = 0
    start_time = time.monotonic()
    current_state = State.WAIT_FOR_BUTTON

# State machine loop
while True:
    current_time = time.monotonic()
# Before anything the system must be started by pressing a button.
    if current_state == State.WAIT_FOR_BUTTON:
        if not button.value: button_reset(State)

# The wheel must be moved towards the input of the scanning system, so an M&M can
be loaded in to the system.
    elif current_state == State.INPUT:
        if not button.value: button_reset(State)
        else:
            set_measure_servo_angle("Input")
            next_state_time = current_time + 0.3
            current_state = State.MEASURE
```

```
# The wheel must move back to the sensor, when an M&M is being picked up.
    elif current_state == State.MEASURE:
        if not button.value: button_reset(State)
        elif current_time >= next_state_time:
            set_measure_servo_angle("Measure")
            next_state_time = current_time + 0.3
            current_state = State.CLASSIFY

# The sensor has to check wethear whether an M&M is present or not, based on this
check the sensor must either go back to the input state, or goes further to the sort
state.
    elif current_state == State.CLASSIFY:
        if not button.value: button_reset(State)
        elif current_time >= next_state_time:
            color = classify_color()
            print(f"Classified color: {color}")
            if color == "Nothing":
                current_state = State.INPUT
            else:
                current_state = State.SORT

# In this state an angle is assigned to the color that was recognized in the classify
state.
    elif current_state == State.SORT:
        if not button.value: button_reset(State)
        else:
            set_sorting_angle(color)
            current_state = State.OUTPUT

# When an angle is assigned the M&M can move towards the output side of the wheel,
where it can go in to one of the 'color boxes'.
    elif current_state == State.OUTPUT:
        if not button.value: button_reset(State)
        else:
            set_measure_servo_angle("Output")
            next_state_time = current_time + 0.3
            current_state = State.DATA

# If a M&M completes the cycle it must be counted, whenever 25 or more M&Ms are
counted the total time it took must be calculated and printed to the console. Then
the machine is being brought back in to the wait for button state.
    elif current_state == State.DATA:
        if not button.value: button_reset(State)
        elif current_time >= next_state_time:
            output_count += 1
            print(f"Output count: {output_count}")
            if output_count >= 25:
                elapsed_time = time.monotonic() - start_time
                print(f"Count reached 25. Time taken: {elapsed_time} seconds")
                current_state = State.WAIT_FOR_BUTTON
            else:
                current_state = State.INPUT

    time.sleep(0.01)  # Small sleep to prevent CPU overload
```